ie UNIVERSITY

INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

MCSBT

Master in Computer Science & Business Technology

john E. Ortega, PhD | Fall 2026 Semester 1 – Week 3 Session 6



Class Plan

Session #	Date	Topic
1	**Sep 28**	AI Foundations and Technology Landscape
2	**Sep 30**	AI Strategy and Business Transformation
3	**Oct 2**	Machine Learning Fundamentals (assignment 1)
4	**Oct 5**	Deep Learning
5	**Oct 7**	NLP and Foundation Models (assignment 2)
6	**Oct 14**	Large Language Models (Quiz 1)
7	**Oct 16**	Retrieval-Augmented Generation (RAG)
8	**Oct 19**	Agentic AI Systems (assignment 3)
9	**Oct 21**	Multimodal AI
10	**Oct 26**	Recommender Systems (Quiz 2)
11	**Oct 28**	Cloud AI, MLOps and LLMOps (assignment 4)
12	**Oct 30**	Responsible AI and Governance
13	**Nov 4**	Emerging Topics (assignment 5)
14-15	**Nov 11 + 13**	Final Presentations

Session 6 Agenda

- LLM Architecture
- Pretraining
- Fine-Tuning
- Alignment
- RLHF
- ~~Constitutional AI~~ Generative AI (Gen AI)
- Reasoning Models
- ~~Small Language Models (SLMs)~~
- ~~Open vs Closed Models~~
- LLM Evaluation

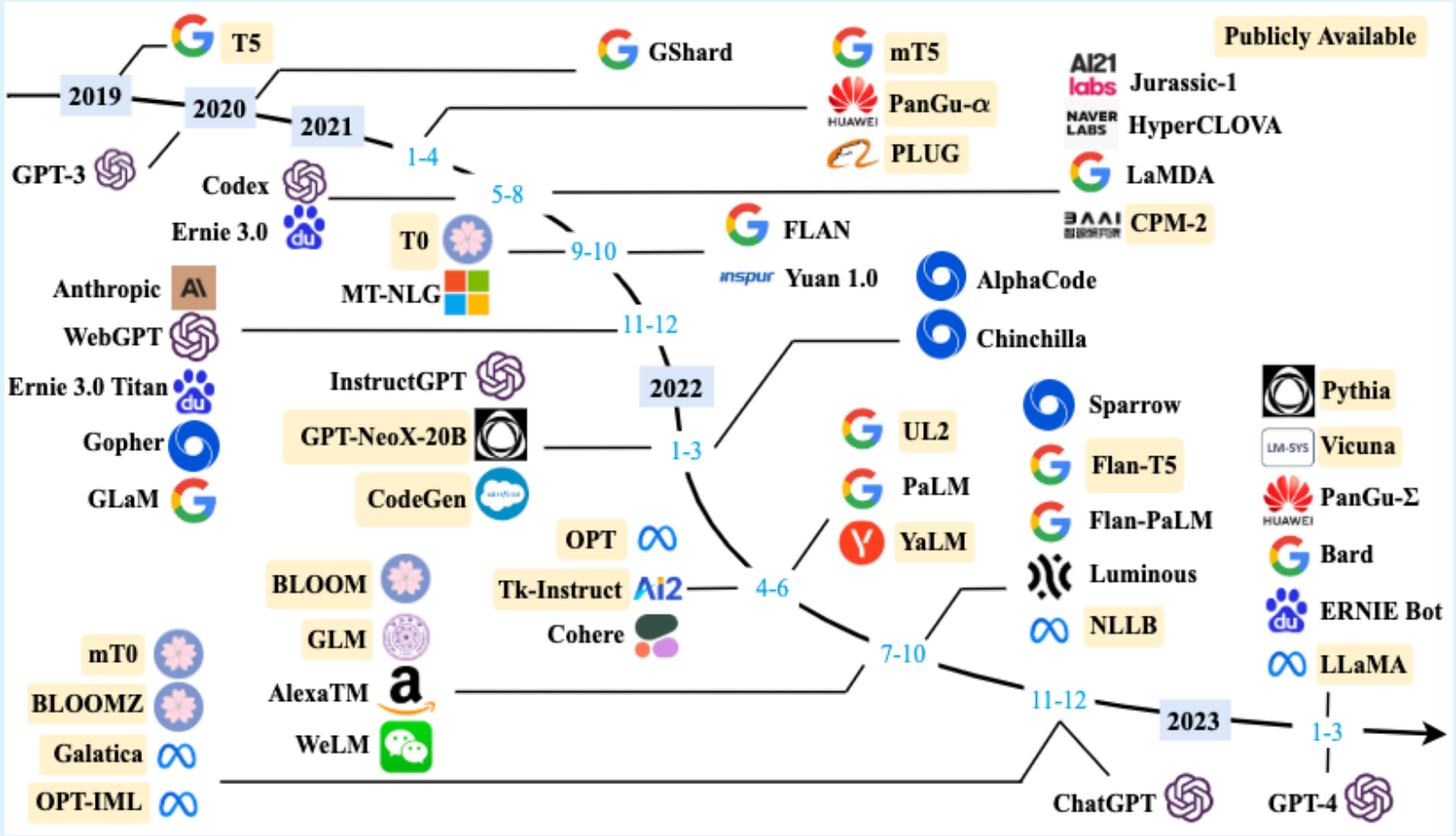


Session 6

Large Language Models

LLM Architecture

Large Language Models



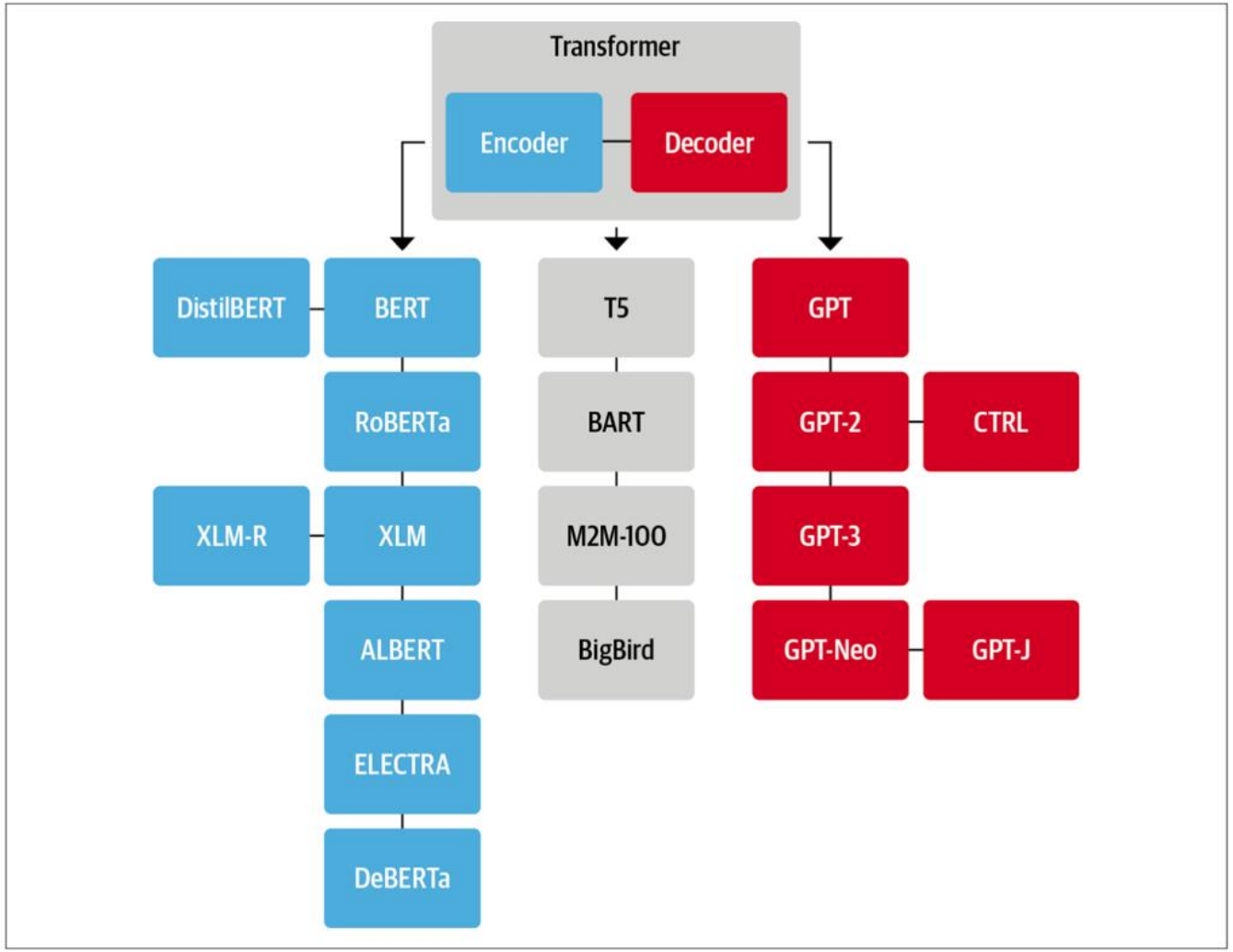
Model	Category	Size
GPT3 [55]	Casual decoder	175B
PanGU- α [73]	Casual decoder	207B
OPT [79]	Casual decoder	175B
PaLM [56]	Casual decoder	540B
BLOOM [66]	Casual decoder	176B
MT-NLG [90]	Casual decoder	530B
Gopher [59]	Casual decoder	280B
Chinchilla [34]	Casual decoder	70B
Galactica [35]	Casual decoder	120B
LaMDA [85]	Casual decoder	137B
Jurassic-1 [89]	Casual decoder	178B
LLaMA [57]	Casual decoder	65B
GLM-130B [80]	Prefix decoder	130B
T5 [71]	Encoder-decoder	11B

Note the number of **Parameters**!

Also Note: **T5** is **Encoder-Decoder**

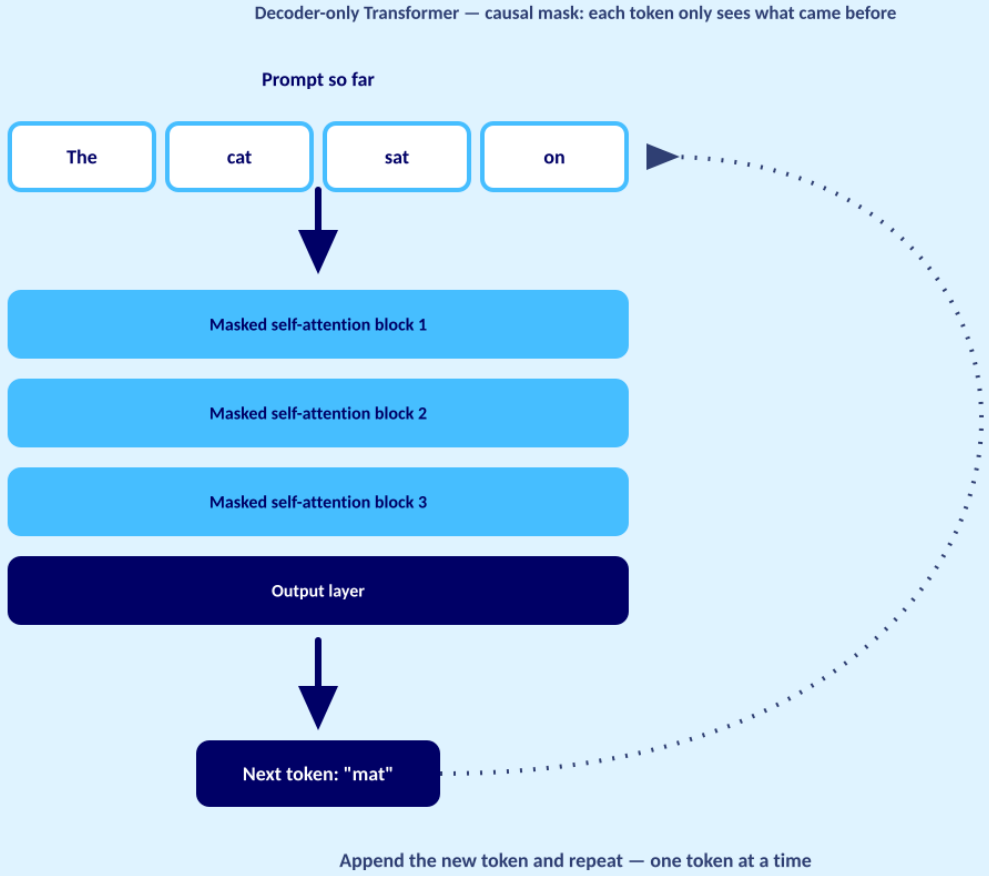
Transformer Models

- Not ALL Transformers are Decoder-Only
- Not ALL LLMs are Transformers
- Not ALL LLMs are Decoder-Only



GPT: Decoder-Only, Left to Right

GPT (OpenAI, from 2018) is a **decoder-only Transformer and Large Language Model** trained with a simple objective: predict the next token given everything before it. Generation repeats this one token at a time.



Sources: Radford et al., "Improving Language Understanding by Generative Pre-Training" (2018) and successors; SLP (Jurafsky & Martin), ch. "Transformers and Large Language Models."

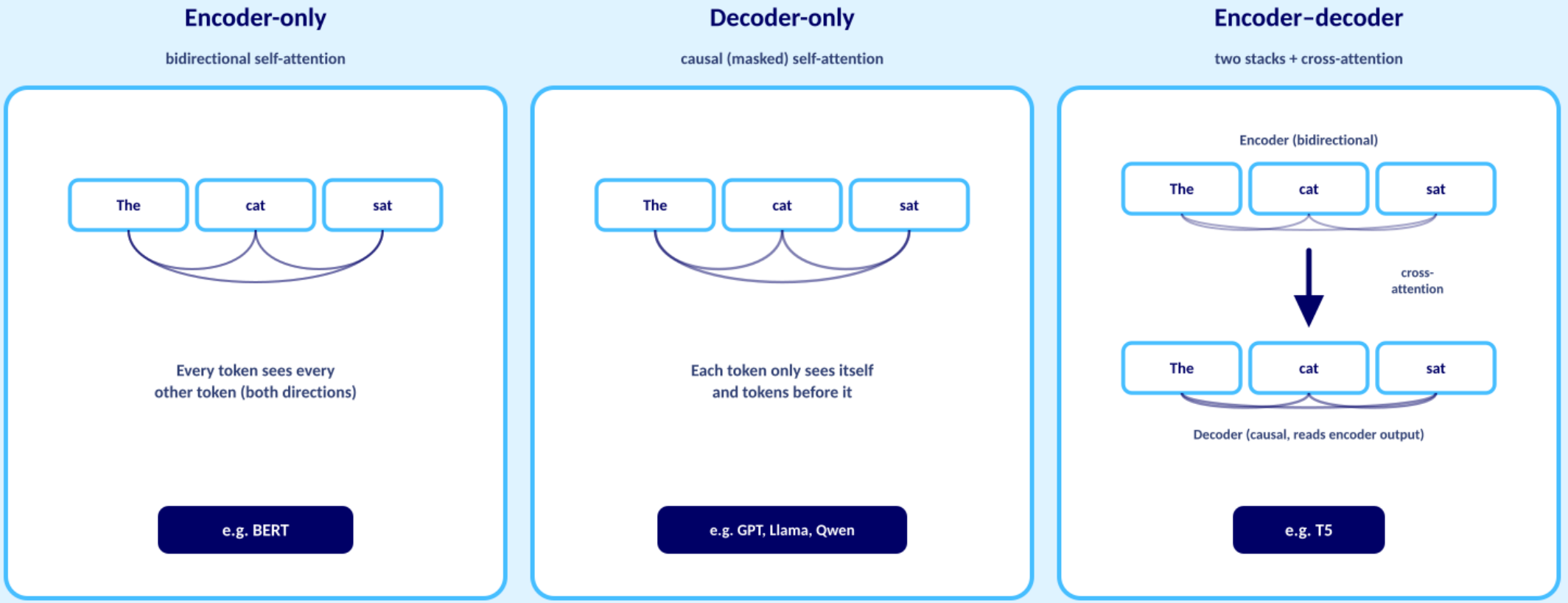
LLMs vs. Transformers

Aspect	Large Language Models (LLMs)	Transformers
Definition	AI models designed to understand and generate natural language	Neural network architecture used to process and learn relationships in data
Purpose	Primarily used for language understanding and generation	Provides an architecture for building models for various tasks
Relationship	Many modern LLMs are built using Transformers	Transformers are commonly used as the architecture behind LLMs
Architecture	Can be based on architectures such as Transformer	Uses self-attention and related components
Training Data	Typically trained on large text datasets	Can be trained on text, images, audio and other types of data
Applications	Text generation, summarization, translation, question answering and coding	NLP, computer vision, speech processing and multimodal AI
Output	Primarily language-based outputs such as text	Depends on the task, such as text, classifications or predictions
Examples	GPT, LLaMA and other large-scale language models	BERT, GPT, T5 and ViT

Shorthand for this course: an LLM is a type of model; a Transformer is the architecture most modern LLMs (and many non-LLM models, e.g. ViT) are built from.

Three Transformer Architectures, Three Jobs

Every model in this session is built from the same self-attention block — what differs is which tokens each token is allowed to attend to, and whether there is one stack or two.



A second axis, independent of this one: dense vs. Mixture-of-Experts. A dense model activates every parameter on every token; an MoE model (Llama 4, Qwen 3, DeepSeek, Mistral Large 3) routes each token through only a few expert sub-networks, so a much larger model costs roughly the same to run per token.

Sources: Vaswani et al. (2017); Devlin et al. (2019); Radford et al. (2018); Raffel et al. (2020); SLP (Jurafsky & Martin), ch. "Transformers and Large Language Models."

The Same Three Types, in PyTorch

torch.nn provides one set of building blocks for all three architectures — what changes is whether a mask is passed, and whether there is one stack or two.

```
import torch
import torch.nn as nn

# 1) Encoder-only (BERT-style) — bidirectional, no mask
enc_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8, batch_first=True)
encoder_only = nn.TransformerEncoder(enc_layer, num_layers=6)
out = encoder_only(x)                          # x: (batch, seq_len, d_model)

# 2) Decoder-only (GPT-style) — causal mask blocks future tokens
dec_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8, batch_first=True)
decoder_only = nn.TransformerEncoder(dec_layer, num_layers=6)
causal_mask = nn.Transformer.generate_square_subsequent_mask(seq_len)
out = decoder_only(x, mask=causal_mask)

# 3) Encoder-decoder (T5-style) — two stacks + cross-attention
seq2seq = nn.Transformer(d_model=512, nhead=8, num_encoder_layers=6,
                         num_decoder_layers=6, batch_first=True)
out = seq2seq(src, tgt, tgt_mask=causal_mask)   # cross-attention is automatic
```

Illustrative PyTorch (torch.nn) — layer counts and d_model/nhead are examples, not a specific published model's exact configuration.

Summary: T5 and Qwen

T5

Google Research · 2020

Architecture: encoder–decoder Transformer

- Sizes: 60M–11B parameters (original); Flan-T5 instruction-tuned successors extend the family
- Pretrained on the C4 corpus with a span-corruption (fill-in-the-blank) objective
- Every task — translation, summarization, classification — is framed as text in, text out

Known for: proving one unified architecture and loss function can match task-specific models across many different NLP tasks.

Qwen 3

Alibaba Cloud · 2025

Architecture: hybrid dense + Mixture-of-Experts

- Dense models 0.6B–32B; MoE models up to 235B total parameters (22B active)
- Trained on 36 trillion tokens across 119 languages; Apache 2.0 license
- Single model can switch between fast “non-thinking” and step-by-step “thinking” modes

Known for: spanning the widest size range of any open-weight family — small enough to self-host, large enough to compete with frontier models.

Sources: Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer” (2020); Alibaba Cloud, Qwen3 release documentation (April 2025).

Summary: DeepSeek and Mistral

DeepSeek V3 / R1

DeepSeek-AI · Dec 2024 – Jan 2025

Architecture: Mixture-of-Experts

- 671B total parameters, only 37B active per token
- Multi-head Latent Attention (MLA) for memory- and compute-efficient training
- R1 adds reinforcement learning on top of V3 to produce extended step-by-step reasoning

Known for: reaching frontier-level performance at a far lower reported training cost, and showing RL — not just more data — can improve reasoning.

Mistral

Mistral AI · 2023–2025

Architecture: dense → sparse MoE evolution

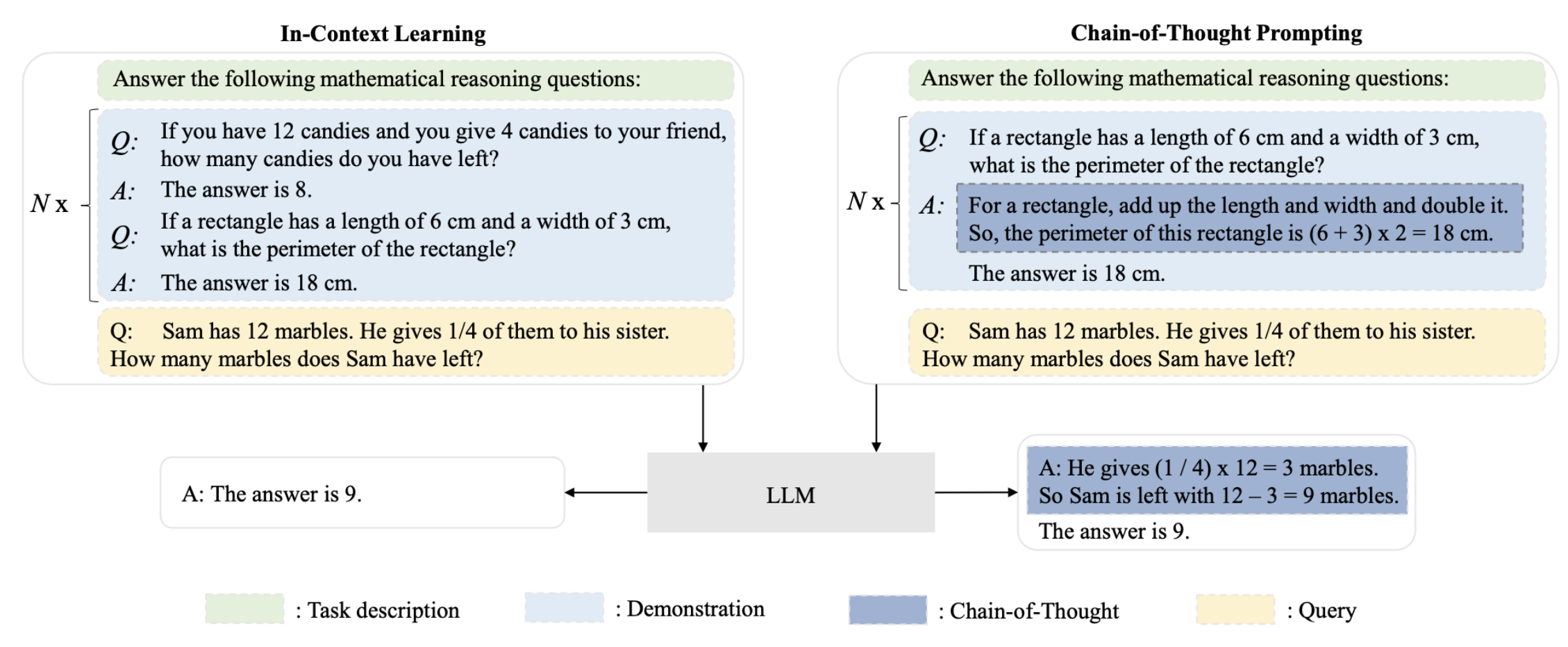
- Mistral 7B (2023): dense, outperformed larger Llama 2 models at a fraction of the size
- Mixtral 8x7B (2023): sparse MoE, 46.7B total / 12.9B active parameters
- Mistral Large 3 (2025): sparse MoE, 675B total / 41B active parameters

Known for: being Europe's leading open-weight lab and an early popularizer of efficient Mixture-of-Experts scaling.

Sources: DeepSeek-AI, "DeepSeek-V3 Technical Report" (2024) and DeepSeek-R1 release (Jan 2025); Mistral AI model releases and changelog (2023–2025). Figures reflect publicly reported specs and may change as newer versions ship.

Large Language Model Tuning vs SLMs

- Emergent abilities of LLMs: the abilities that are not present in smaller models but are present in larger models.





Session 6

Large Language Models

Pre-Training

Pretraining: Learning Language Before Any Task

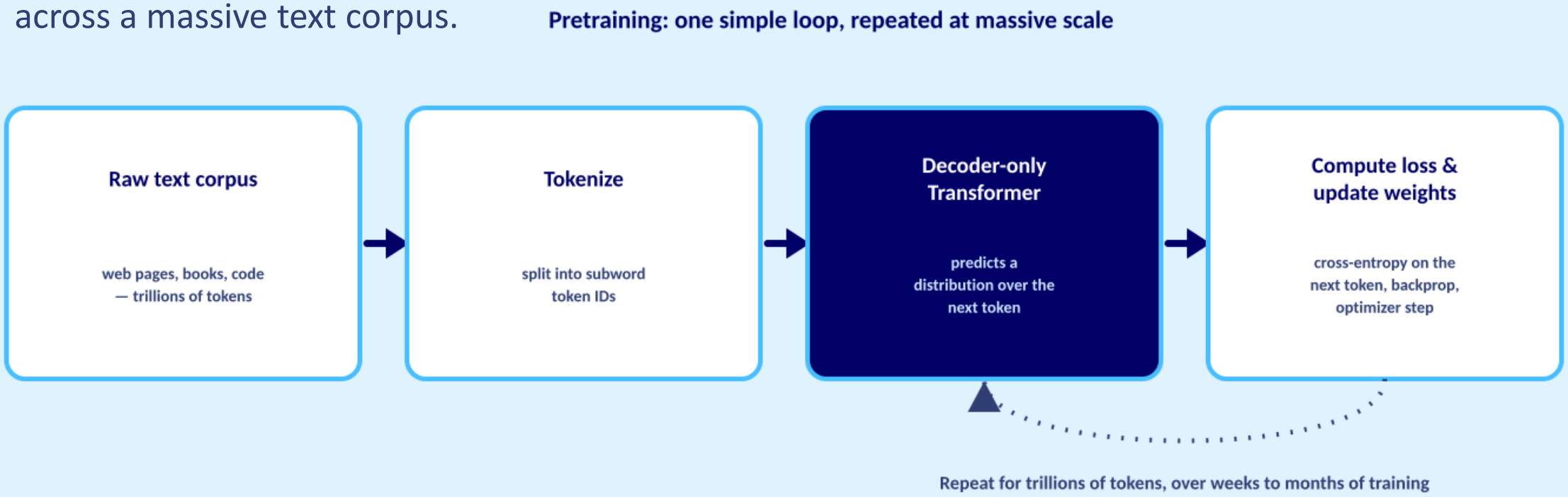
Pretraining is the first and most expensive stage of building an LLM: a decoder-only Transformer learns from massive amounts of raw, unlabeled text using one simple self-supervised objective — predict the next token.

- **Self-supervised, not supervised:** The training signal is the text itself — the next word is the label — so no human annotation is needed at this stage.
- **Objective:** Minimize cross-entropy loss between the model's predicted next-token distribution and the actual next token, at every position, in every sequence.
- **Scale:** Modern LLMs pretrain on trillions of tokens (web text, books, code) across thousands of GPUs/TPUs, for weeks to months.
- **Output:** A "base" or "foundation" model with broad language ability and world knowledge — not yet aligned to follow instructions; that comes later via fine-tuning and RLHF.

Sources: Radford et al., "Improving Language Understanding by Generative Pre-Training" (2018); Kaplan et al., "Scaling Laws for Neural Language Models" (2020); SLP (Jurafsky & Martin), ch. "Transformers and Large Language Models."

The Pre-Training Pipeline

The same loop — predict the next token, measure the error, update the weights — runs over and over across a massive text corpus.



Scale matters more than cleverness here: per the Chinchilla/Kaplan scaling laws, loss improves predictably as data, parameters, and compute all grow together — which is why pretraining runs are measured in GPU-years, not GPU-hours.

Sources: Kaplan et al. (2020); Hoffmann et al., "Training Compute-Optimal Large Language Models" (2022, the "Chinchilla" paper); SLP (Jurafsky & Martin).

A Minimal LLM Pretraining Loop, in PyTorch

Strip away the scale (fewer GPUs, a smaller model) and every LLM's pretraining loop looks like this: predict the next token, compute the loss, update the weights.

```
import torch
import torch.nn as nn

vocab_size, d_model, seq_len = 50000, 512, 1024
embed   = nn.Embedding(vocab_size, d_model)
decoder = nn.TransformerEncoder(
    nn.TransformerEncoderLayer(d_model=d_model, nhead=8, batch_first=True), num_layers=12)
lm_head = nn.Linear(d_model, vocab_size)

params = list(embed.parameters()) + list(decoder.parameters()) + list(lm_head.parameters())
optimizer = torch.optim.AdamW(params, lr=3e-4)
loss_fn = nn.CrossEntropyLoss()
causal_mask = nn.Transformer.generate_square_subsequent_mask(seq_len - 1)

# Pretraining objective: predict the next token at every position, in parallel
for batch in dataloader:                # batch: (B, seq_len) token ids — no labels needed
    inputs, targets = batch[:, :-1], batch[:, 1:]       # shift by one token
    hidden = decoder(embed(inputs), mask=causal_mask)
    logits = lm_head(hidden)                             # (B, seq_len-1, vocab_size)
    loss = loss_fn(logits.reshape(-1, vocab_size), targets.reshape(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Illustrative PyTorch (torch.nn) — real pretraining adds gradient accumulation, mixed precision, learning-rate warmup/decay, checkpointing, and runs across many GPUs in parallel; the core loop above is unchanged.



Session 6

Large Language Models

Fine-Tuning

Fine-Tuning: How Much of the Model Do You Update?

Pretraining gives a model broad language ability from raw text; fine-tuning adapts that pretrained ("base") model to a narrower task or behavior using a smaller, curated dataset — and the first choice is how much of it to touch.

Full Fine-Tuning

- Every weight in the pretrained model is updated during training
- Highest quality ceiling, but needs a full copy of the model stored per task
- Expensive: large memory, compute, and storage footprint

Parameter-Efficient Fine-Tuning (PEFT)

- Base weights stay frozen; only a small set of new parameters is added and trained
- E.g., LoRA inserts small low-rank matrices next to existing weight matrices
- Trains ~0.1–1% of parameters, at a fraction of the memory and storage cost

Same model, different cost: both update the same pretrained model — the difference is how many parameters change, not what the model is being taught. That's the next axis.

Sources: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models" (2021); Houlsby et al., "Parameter-Efficient Transfer Learning for NLP" (2019).

Fine-Tuning: What Is It Being Taught?

Independent of how much of the model changes, fine-tuning also differs in what signal drives the update — and production LLMs typically apply both of the following, in sequence.

Instruction Fine-Tuning (SFT)

- Supervised examples of (instruction → ideal response) pairs
- Teaches the model to follow instructions and answer in a desired format
- Usually the first fine-tuning stage applied to a pretrained base model

Preference / Alignment Fine-Tuning

- Trained on human preference comparisons — "response A is better than B" — not single correct answers
- Methods: RLHF (reward model + PPO) or DPO (optimizes directly on preference pairs)
- Shapes tone, helpfulness, and safety; usually applied after instruction tuning

Putting it together: pretrain → instruction fine-tune → preference-tune — and PEFT (e.g., LoRA) can be used at any of these stages to keep cost down. The code on the next slide combines exactly that: LoRA + instruction data.

Sources: Ouyang et al., "Training language models to follow instructions with human feedback" (2022); Rafailov et al., "Direct Preference Optimization" (2023).

Fine-Tuning with LoRA, in PyTorch

Putting both axes together: freeze the pretrained weights (PEFT), add a small trainable low-rank update, and train it on instruction data (SFT).

```
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base                        # frozen pretrained layer
        for p in base.parameters(): p.requires_grad = False
        in_f, out_f = base.in_features, base.out_features
        self.A = nn.Parameter(torch.randn(r, in_f) * 0.01)   # new, small
        self.B = nn.Parameter(torch.zeros(out_f, r))          # new, small
        self.scale = alpha / r

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale

# Swap target nn.Linear layers in the pretrained model with LoRALinear
lora_params = [p for n, p in model.named_parameters() if '.A' in n or '.B' in n]
optimizer = torch.optim.AdamW(lora_params, lr=1e-4)   # ~0.1–1% of all parameters
loss_fn = nn.CrossEntropyLoss()

for batch in instruction_dataloader:          # (instruction, response) pairs
    logits = model(batch['input_ids']).logits
    loss = loss_fn(logits.reshape(-1, logits.size(-1)), batch['labels'].reshape(-1))
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Illustrative PyTorch — a real LoRA setup typically uses a library (e.g., Hugging Face peft) to attach adapters automatically across many layers; the underlying idea — freeze base weights, train a small low-rank add-on — is unchanged.



Session 6

Large Language Models

Alignment

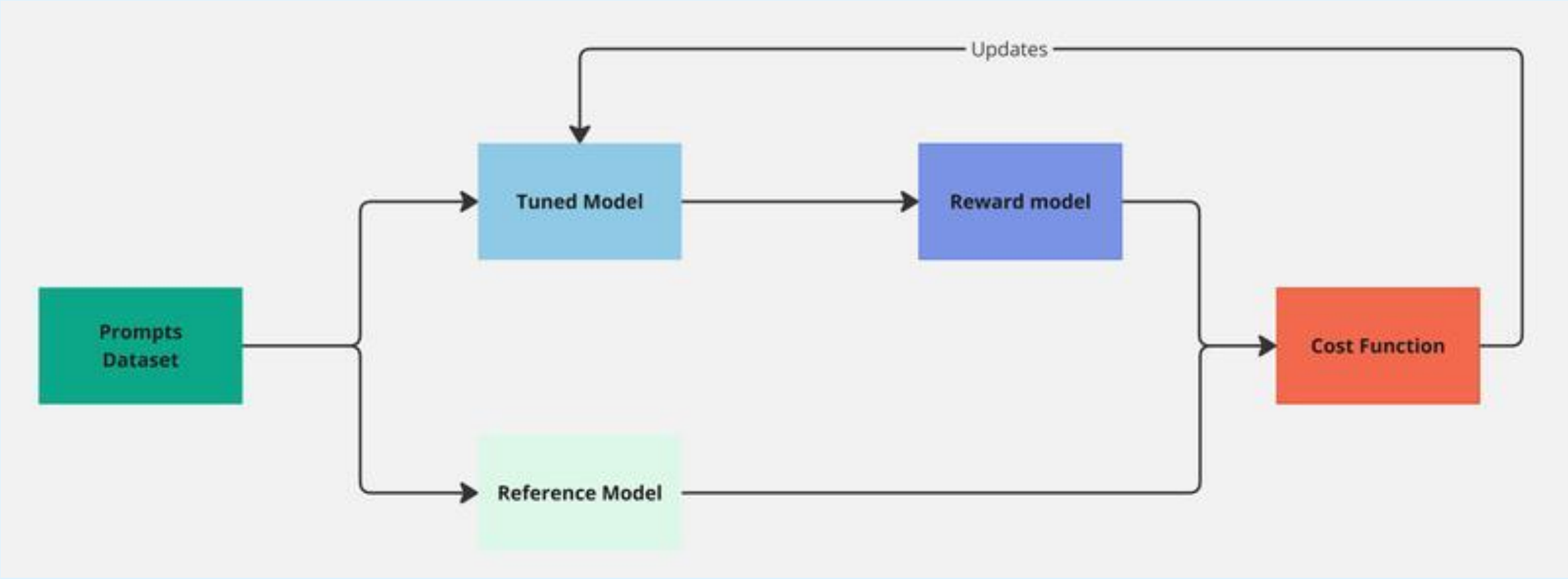
Alignment: Making LLMs Helpful, Honest, and Harmless

Preference fine-tuning (RLHF, DPO) is the mechanism; alignment is the goal — getting a model's behavior to match human intentions and values, not just its ability to follow instructions.

- **The "HHH" objective:** Helpful, Honest, and Harmless — three goals that can trade off: a model that refuses everything is harmless but not helpful
- **RLHF in practice:** Humans rank model responses → a reward model learns to predict those preferences → the LLM is optimized against that reward signal with reinforcement learning (PPO)
- **DPO: a simpler alternative:** Skips the separate reward model and RL loop entirely — optimizes the LLM directly on preference pairs with one supervised-style loss
- **Open challenges:** Reward hacking (gaming the reward model instead of truly improving), over-refusal, and scalable oversight — supervising a model on tasks humans can't easily judge themselves

Sources: Christiano et al., "Deep Reinforcement Learning from Human Preferences" (2017); Ouyang et al. (2022, InstructGPT); Bai et al., "Training a Helpful and Harmless Assistant with RLHF" (2022); Rafailov et al. (2023, DPO).

Alignment: Making LLMs Helpful, Honest, and Harmless





Session 6

Large Language Models

RLHF

RLHF: Turning Human Preferences into a Training Signal

Reinforcement Learning from Human Feedback chains three stages to convert "humans prefer this response" into gradient updates for the LLM.

1. Collect Comparisons

Human labelers rank several candidate responses to the same prompt, from best to worst

2. Train a Reward Model

A separate model learns to predict that ranking — scoring any response the way a human would

3. Optimize the Policy

The LLM (the "policy") is updated via RL — typically PPO — to raise its reward-model score

A KL penalty keeps the policy close to the original (SFT) model: this stops it drifting into outputs that score well on the reward model but no longer sound like a reasonable response — a failure mode called reward hacking.

Sources: Christiano et al., "Deep Reinforcement Learning from Human Preferences" (2017); Ouyang et al., "Training language models to follow instructions with human feedback" (2022).

A Simplified RLHF Update, in PyTorch

Score each sampled response with the reward model, penalize drifting from the reference (SFT) model, then take a clipped policy-gradient step — the core of PPO.

```
import torch

kl_coef, clip_eps = 0.02, 0.2

for batch in prompts_dataloader:                        # just prompts, no labels needed
    responses, logp, ref_logp = rollout(policy, ref_model, batch)   # sample + score
    reward = reward_model(batch, responses)             # scalar preference score per response
    kl = (logp - ref_logp).sum(-1)                       # drift from the original SFT model
    advantage = reward - kl_coef * kl                    # penalize straying too far from ref_model

    old_logp = logp.detach()                             # logprobs when the rollout was sampled
    new_logp = policy.log_prob(batch, responses)         # recompute under the current policy
    ratio = torch.exp(new_logp.sum(-1) - old_logp.sum(-1))
    clipped_ratio = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps)
    loss = -torch.min(ratio * advantage, clipped_ratio * advantage).mean()  # PPO-clip

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Illustrative PyTorch — real RLHF implementations (e.g., TRL) add batched generation, value/critic networks, reward normalization, and run across many GPUs; the core clipped-objective idea above is unchanged.

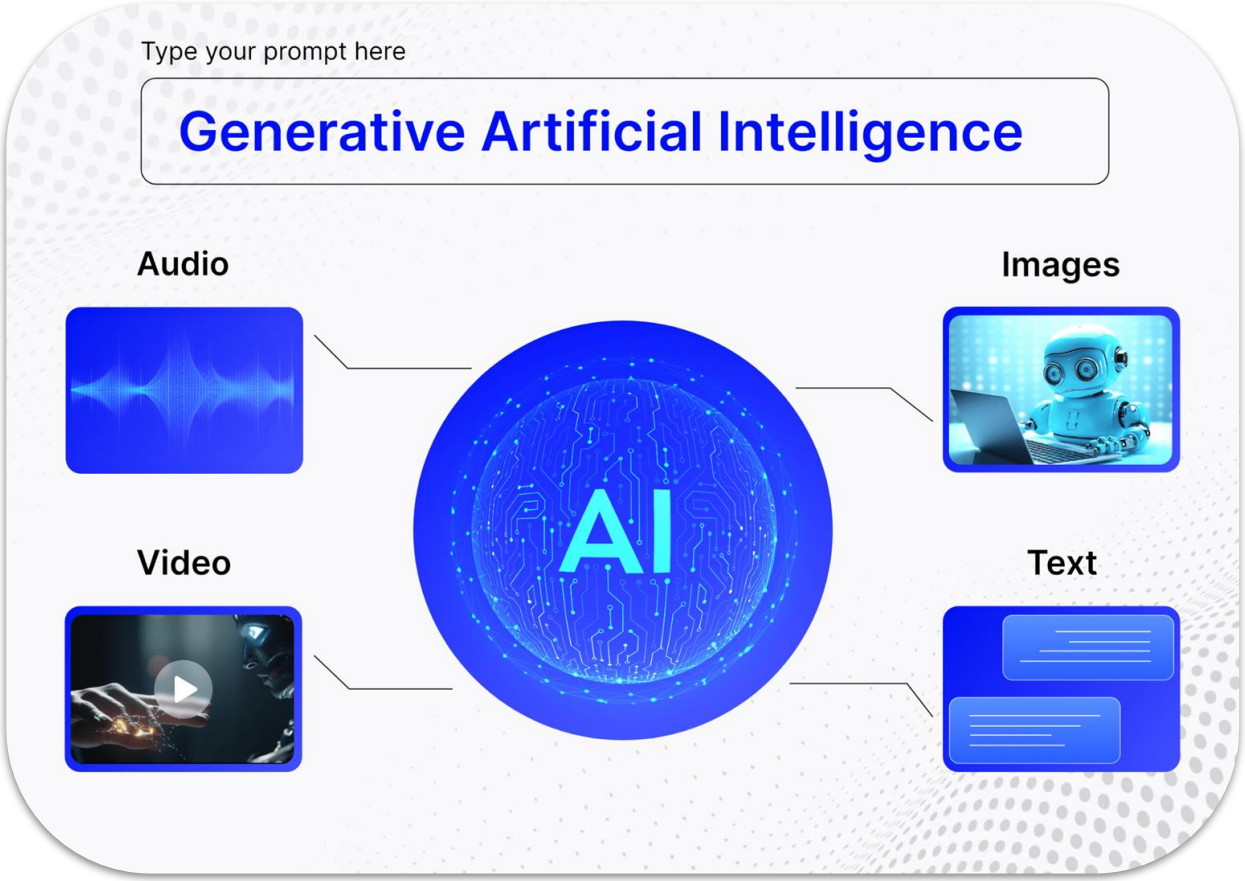


Session 6

Large Language Models

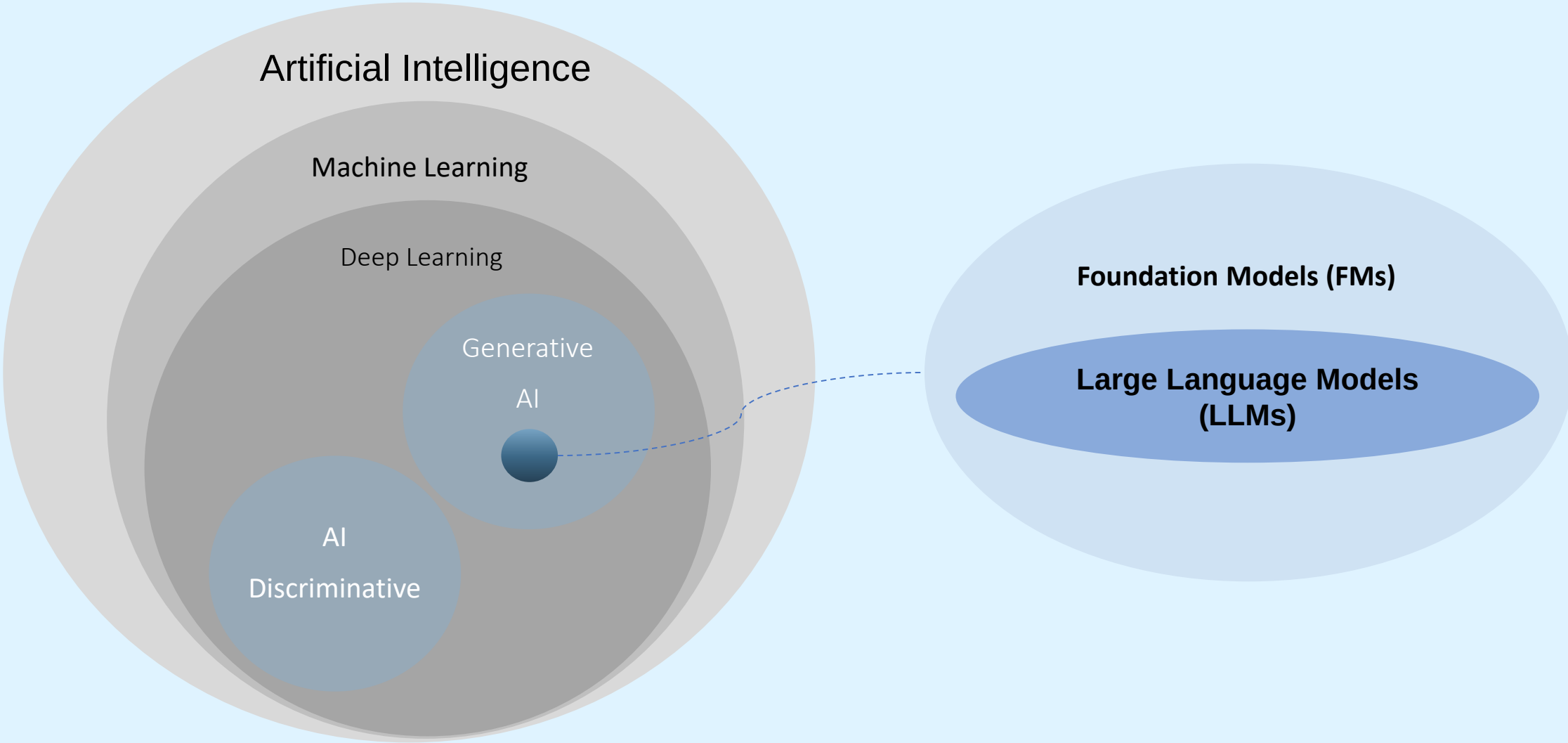
Gen AI

What is Generative AI (Gen AI)?



"A generative artificial intelligence system, is a system capable of **creating text, images, or other content** (music, video, voice, etc.) based on an instruction from a human user. These systems produce new content from training data. » CNIL (May 2023)

What is Gen AI?



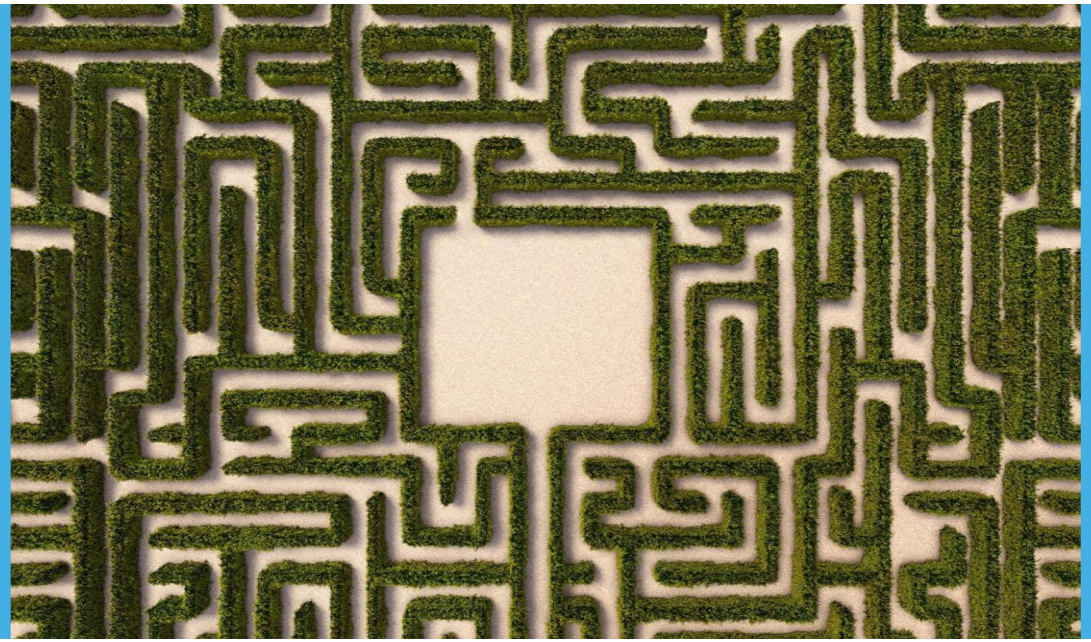
How is Gen AI Different?

Traditional AI



Simple model :
Purpose-built for one use

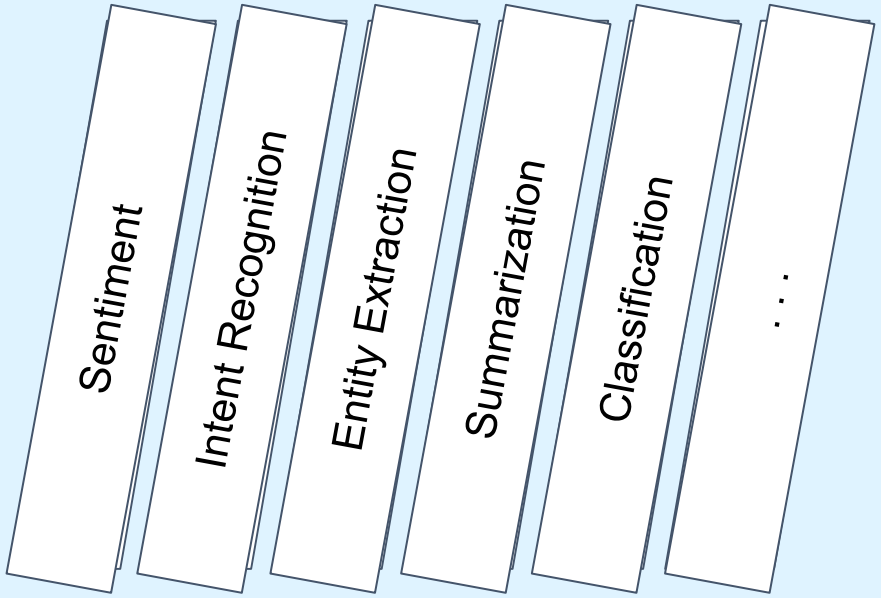
Foundation models



Foundation models :
More generalized intelligence
with many applications

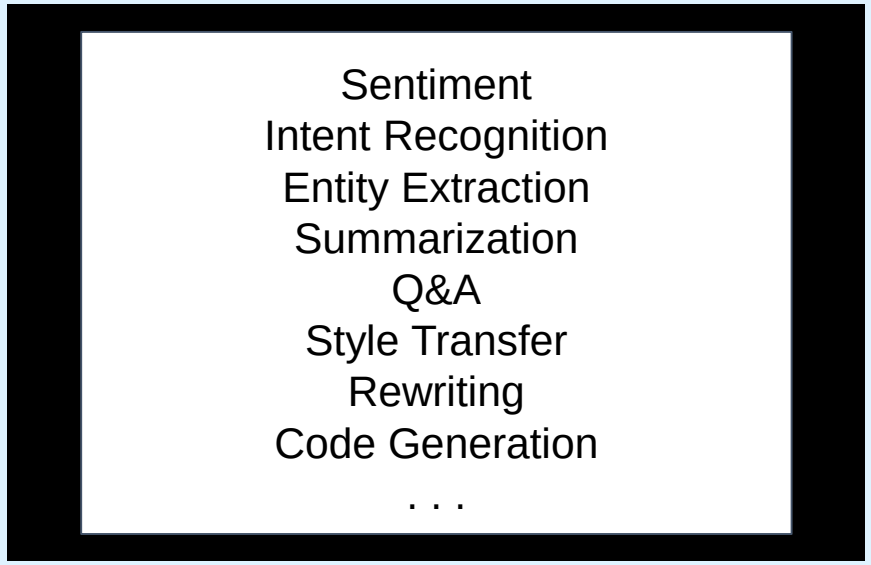
How is Gen AI Different?

Classic Language / AI Models



- One model per “skill”
- Plenty of data required for training
- Highly focused/optimized for use case

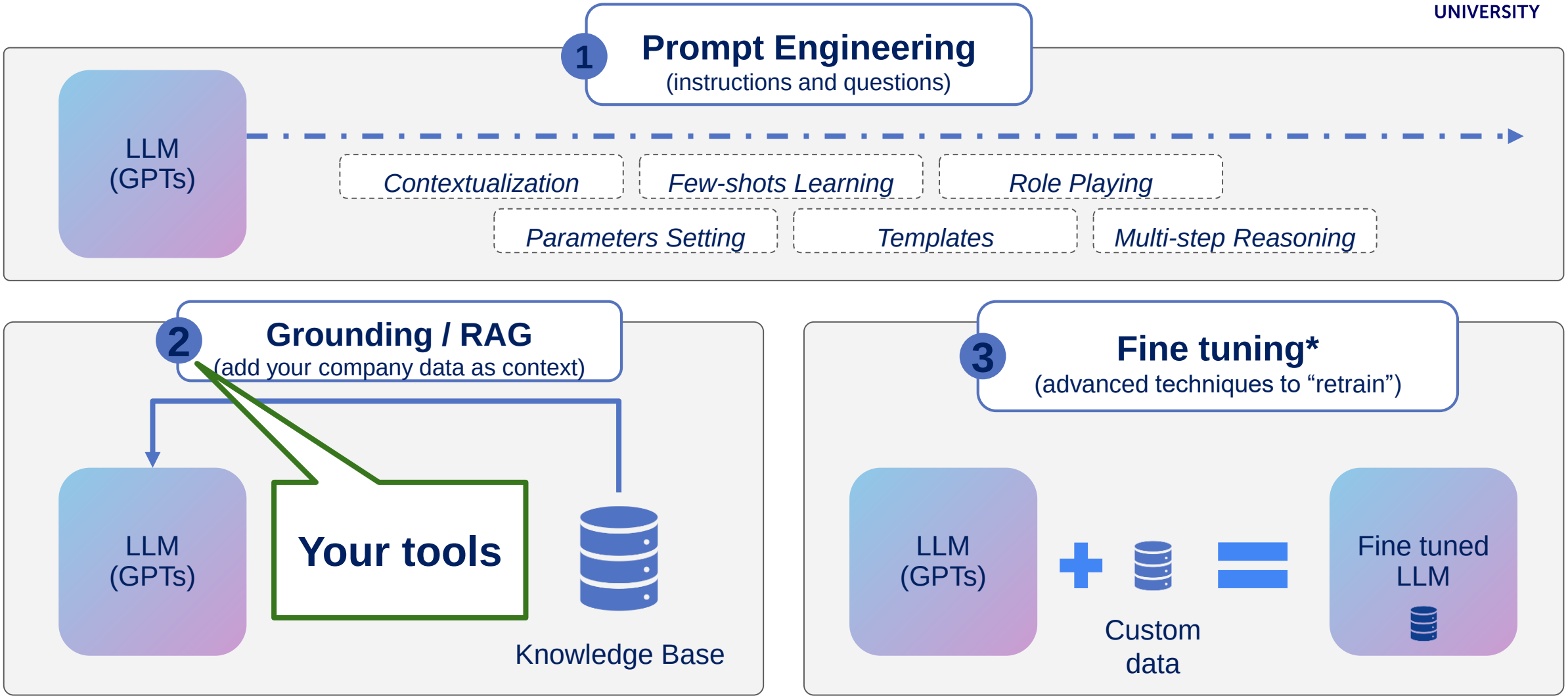
Foundation Models / Large Language Models (LLM)



- A single model for ALL use cases
- Large model already pre-trained with data
- Instruction in “human language”

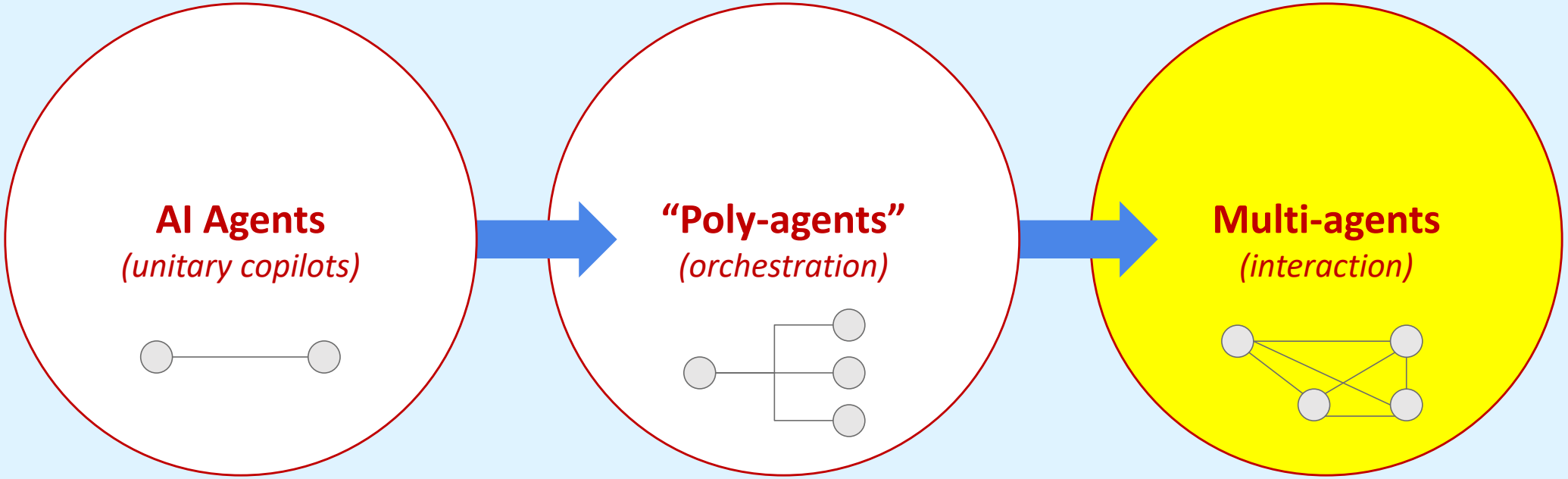
How do we use Pre-Trained Language Models (PLMs)

ie UNIVERSITY

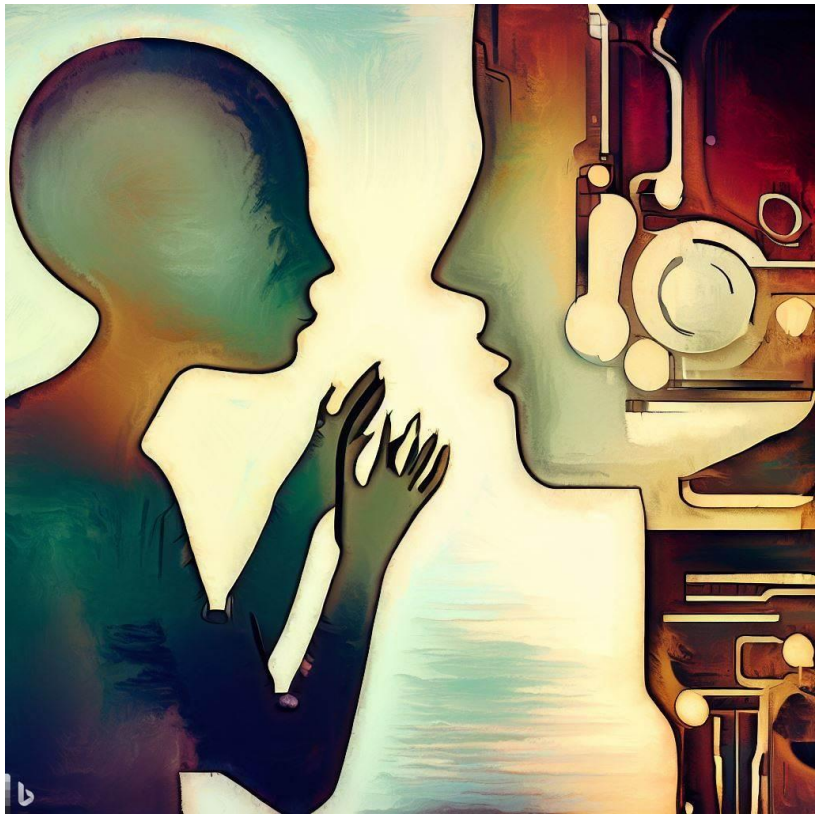


** only if steps 1 and 2 are not enough to reach your performance goals*

Gen AI and Agents



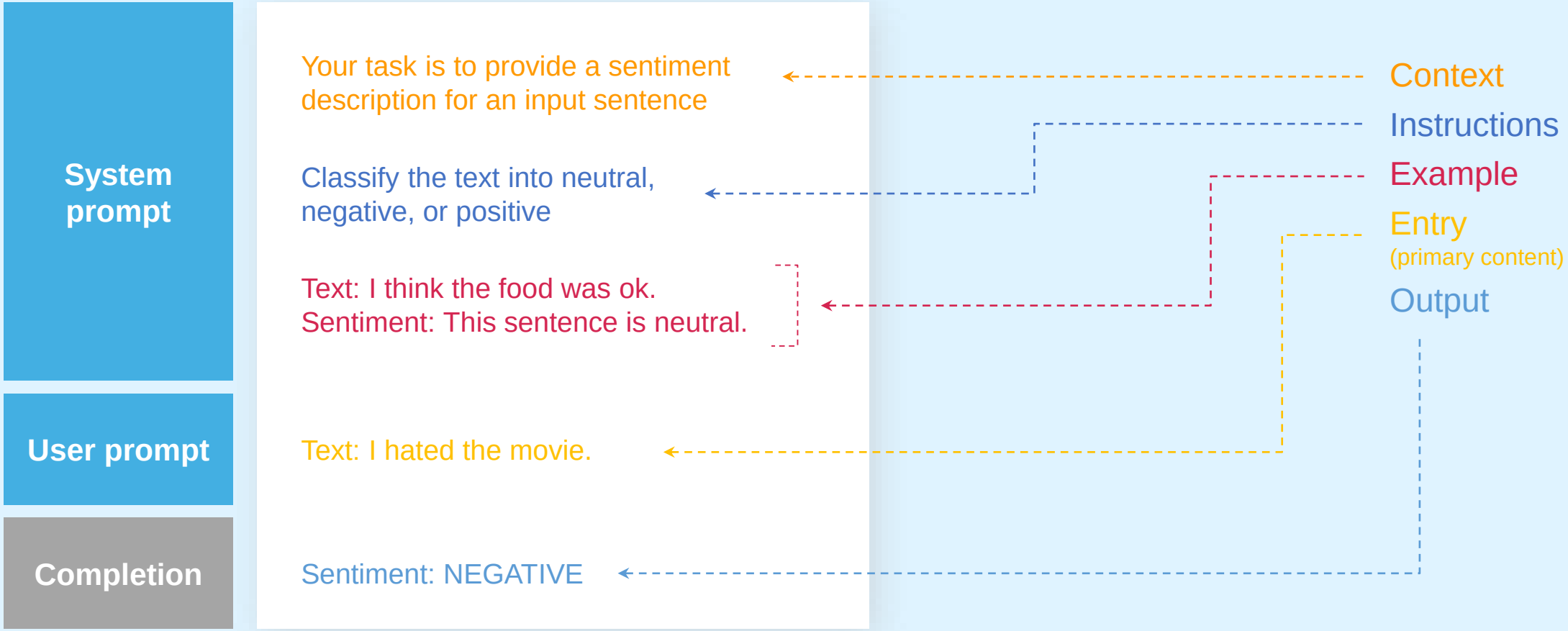
Agent Communication



- Prompt engineering is a concept in natural language processing (NLP) that involves embedding task descriptions (prompts) in the input to prompt the model to produce the desired results (completions).

- A prompt typically includes descriptions of problems, instructions on how to solve them and, possibly, examples of correct problem-solution pairs.

Prompting and Completeness



Agent Communication

PROMPT	RESPONSE
Write a tagline for an ice cream shop.	We serve up smiles with every scoop!
Table customers, columns = [CustomerID, FirstName, LastName, Company, Address, City, State, Country, PostalCode] Create a SQL query for all customers in Texas name Jane query =	SELECT * FROM customers WHERE State = 'TX' AND FirstName = 'Jane'
A ball of fire with vibrant colors to show the speed of innovation at our media and entertainment company.	

Prompt Engineering Example

Prompt example

> Generate a corporate presentation. Start with context review, then talk about workstreams, and action items.

Prompt example (advanced)

> Generate a corporate presentation. Follow this structure, one page per topic

- Start with context review, based on information from finance industry websites
- Workstreams and details, use table format
- Action items

Include all the sources.



Session 6

Large Language Models

Reasoning Models

Reasoning Models: Thinking Before Answering

A standard LLM generates an answer directly. A reasoning model first generates an extended chain of intermediate reasoning — then answers — trading extra inference-time compute for better accuracy on hard problems.

- **Chain-of-thought (CoT):** The model writes out intermediate reasoning steps before its final answer, instead of jumping straight to a conclusion
- **Test-time compute:** Letting the model "think longer" — generate more reasoning tokens — improves accuracy on math, code, and logic: a new scaling axis alongside model size and training data
- **Trained, not just prompted:** Reasoning models are explicitly trained to produce long, useful chains of thought, rather than relying on a user's "think step by step" prompt
- **Where it helps most:** Multi-step math, coding, and logical puzzles see the biggest gains; simple factual lookups or casual conversation see little benefit

Sources: Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (2022); OpenAI, "Learning to Reason with LLMs" / o1 system card (2024); Snell et al., "Scaling LLM Test-Time Compute Optimally" (2024).

Training Reasoning Models: Reward What's Verifiable

Reasoning models build on the fine-tuning and RLHF pipeline already covered — with one key twist in where the reward signal comes from.

- **RL on verifiable rewards (RLVR):** For math and code, the reward can be computed automatically — does the final answer match, do the unit tests pass — so large-scale RL no longer needs a human to rank every example
- **Emergent long reasoning:** DeepSeek-R1 showed that large-scale RL on verifiable rewards — with minimal supervised fine-tuning — can lead a base model to spontaneously develop long chains of thought, self-checking, and backtracking
- **A fast/thinking tradeoff:** Reasoning tokens add latency and inference cost per query, so many systems expose both a fast mode and a "thinking" mode — as seen earlier with Qwen3 — letting the model (or the user) choose how long to think

Sources: DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (2025); OpenAI o1 system card (2024).

Reasoning Models in Practice

Every major lab now ships a model with an explicit "thinking" mode — the same RL-on-verifiable-rewards idea, under a different name.

Model	Lab	What's Distinctive
OpenAI o1 / o3	OpenAI	First mainstream reasoning LLMs; graduated "thinking effort" levels, chain-of-thought hidden from the user
DeepSeek-R1	DeepSeek	Open-weights, with an openly published training recipe showing reasoning emerging from large-scale RL alone
Qwen3 (Thinking Mode)	Alibaba	A single model toggles between a fast "non-thinking" mode and a step-by-step thinking mode
Gemini 2.5	Google DeepMind	An adjustable "thinking budget" that controls how many reasoning tokens the model spends per query
Claude (Extended Thinking)	Anthropic	Visible reasoning traces that interleave step-by-step thinking with tool calls
Grok 4 (Think Mode)	xAI	A reasoning mode trained via large-scale RL on verifiable, auto-gradable rewards

Sources: OpenAI o1/o3 system cards; DeepSeek-AI, "DeepSeek-R1" (2025); Alibaba Cloud, Qwen3 documentation; Google DeepMind, Gemini 2.5 model card; Anthropic, Claude extended thinking documentation; xAI, Grok 4 announcement.



Session 6

Large Language Models

LLM Evaluation

Evaluating LLMs: What Are We Measuring?

As LLMs move into production, evaluation shifts from simple accuracy toward a wider set of qualities — above all, whether an answer can be trusted.

- **Hallucination:** The model states something confidently that is factually wrong or unsupported by any source — the core trust risk in any user-facing deployment
- **Faithfulness (groundedness):** For RAG and tool-using systems: is the generated answer actually supported by the retrieved context or documents provided — not just plausible-sounding
- **Relevance & completeness:** Does the answer address the actual question and cover what's needed, without padding, omission, or going off-topic
- **Safety & toxicity:** Does the output avoid harmful, biased, or policy-violating content — typically scored alongside the other three, not instead of them

Sources: Ji et al., "Survey of Hallucination in Natural Language Generation" (2023); Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation" (2023).

Hallucination is the main problem



Source : Bing Create

- **Hallucinations** for generative AI are outputs that are not real, do not correspond to any data on which the algorithm has been trained or to any other identifiable model.
- For example, a chatbot that says something false or irrelevant.

Hallucination Avoidance: Problematic Breakdown

1. Give instructions as clearly as possible
2. Divide complex tasks into simpler sub-tasks
3. Structure the instruction to keep the model focused on the task
4. Ask the model to explain before answering
5. Ask for justifications for many possible answers, then summarise
6. Generate many outputs, then use the model to choose the best
7. Customise models to maximize performance

Evaluation Tools: Open-Source and Commercial

A growing ecosystem of tools automates these checks — some free and self-hosted, some managed platforms with dashboards and alerting.

Tool	Type	Focus
RAGAs	Open-source	Automated RAG metrics — faithfulness, answer relevancy, context precision/recall
DeepEval	Open-source	Pytest-style unit testing for LLM outputs, including hallucination and bias checks
H2O Eval Studio	Open-source (H2O.ai)	Benchmarks and compares LLMs/RAG pipelines on hallucination, leakage, and custom tests
TruLens	Open-source	Tracing plus "feedback functions" that score groundedness and relevance in production
Galileo	Commercial	Real-time hallucination detection and observability for production LLM applications
LangSmith	Commercial (LangChain)	Tracing, dataset-based evaluation runs, and human annotation queues

Sources: ragas.io; deepeval (Confident AI) documentation; H2O.ai Eval Studio documentation; trulens.org; Galileo and LangSmith product documentation.

Evaluating RAG Outputs with RAGAs (Open Source)

RAGAs scores the two risks from the first slide directly: faithfulness (is the answer grounded in its retrieved context?) and relevancy (does it address the question?).

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision
from datasets import Dataset

# One row per (question, retrieved context, generated answer, ground truth)
data = Dataset.from_dict({
    "question":     ["What year was the Transformer paper published?"],
    "contexts":     [["Attention Is All You Need was published in 2017."]],
    "answer":       ["The Transformer paper was published in 2017."],
    "ground_truth": ["2017"],
})

results = evaluate(
    dataset=data,
    metrics=[faithfulness, answer_relevancy, context_precision],
)

print(results)
# {'faithfulness': 1.00, 'answer_relevancy': 0.97, 'context_precision': 1.00}

low_faithfulness = results.to_pandas().query("faithfulness < 0.7")
# Flag any answer not well-supported by its retrieved context for human review
```

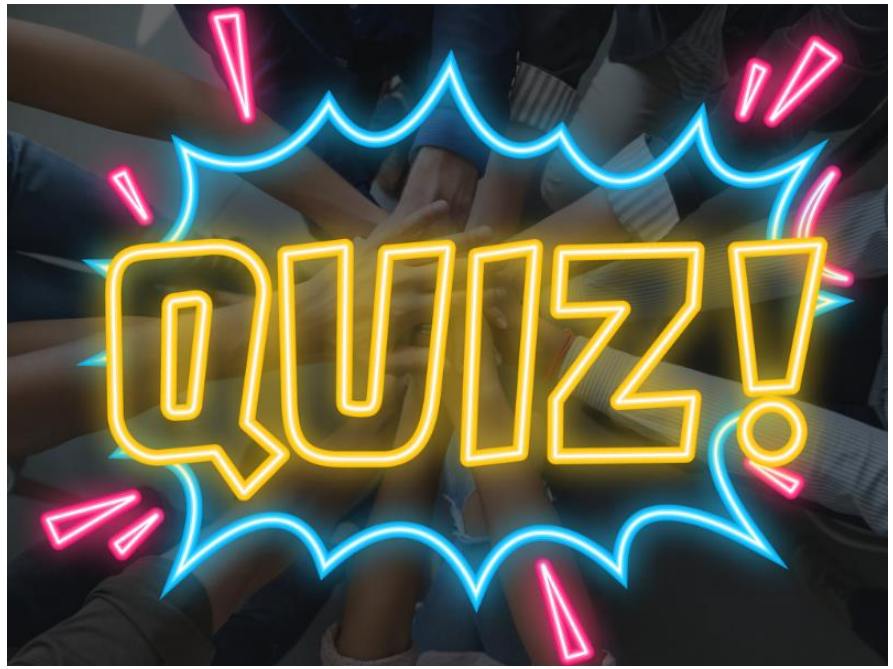
Sources: ragas.io documentation. H2O.ai's H2O Eval Studio provides a comparable open-source workflow for benchmarking hallucination and groundedness across models and RAG pipelines.

Session 6

Quiz 1

Quiz 1 – In Class Quiz

- Multiple Choice
- 20 minutes at the end of class
- Completed on Blackboard
- Covers all materials until today
- Covers Assignments
- Covers Textbook information
- Open Book
- No Group Conversations
- 6 Questions
- 10 points total (counted as a percentage)



Thank you

Questions & discussion

John E. Ortega, Phd

johnevano@faculty.ie.edu

Office hours: By Appointment via Zoom

